# generalManagement HTTP/TLS/JSON

## Interface Design Description

Service ID: *"generalManagement"*

**Abstract**
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the DynamicServiceOrchestration system.

# Contents

**1 Overview** **3**
1.1 Interface registration . . . 3

**2 Service Operations** **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

**3 Data Models** **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

**4 References** **8**

**5 Revision History** **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

# 1 Overview

This document describes the `generalManagement` service interface of the DynamicServiceOrchestration system, which provides general management of the DynamicServiceOrchestration system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

| Profile type | Type | Version |
|---|---|---|
| Transfer protocol | HTTP | 1.1 [1] |
| Data encryption | TLS | 1.3 [2], 1.2 [3] |
| Encoding | JSON | RFC 8259 [4] |
| Compression | N/A | - |
| Semantics | N/A | - |
| Ontology | N/A | - |

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

## 1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8441 and base path `/serviceorchestration/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

| Operation | Method | Path |
|---|---|---|
| `get-log` | POST | `/logs` |
| `get-config` | GET | `/get-config` |

# 2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

## 2.1 POST /serviceorchestration/general/mgmt/logs

**Operation:** get-log
**Input:** LogRequest
**Output:** LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceorchestration/general/mgmt/logs HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

## 2.2 GET /serviceorchestration/general/mgmt/get-config?keys=*{keys}*

**Operation:** get-config
**Output:** KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /serviceorchestration/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: serviceorchestration-dynamic:8441
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

# 3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

## 3.1 struct ErrorMessage

| Object Field | Value Type | Description |
|---|---|---|
| "errorMessage" | String | |
| "errorCode" | Number | |
| "exceptionType" | ExceptionType | |
| "origin" | String | |

## 3.2 struct KeyValues

Configuration values of the system.

| Object Field | Value Type | Description |
|---|---|---|
| "map" | Object<String> | The configuration keys and values; secrets are not returned. |

## 3.3 struct LogEntry

One log entry.

| Object Field | Value Type | Description |
|---|---|---|
| "logId" | String | Identifier of the log entry. |
| "entryDate" | DateTime | UTC time of the log entry. |
| "logger" | String | Logger (Java class) that wrote the entry; as filter, its name. |
| "severity" | String | Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level. |
| "message" | String | Text of the log entry or status message of the job. |
| "exception" | String | Stack trace or exception text of the log entry, if any. |

## 3.4 struct LogEntryListResponse

List of log entries.

| Object Field | Value Type | Description |
|---|---|---|
| "entries" | Array<LogEntry> | Matching records of the current page. |
| "count" | Number | Total number of matching records (all pages). |

## 3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

| Object Field | Value Type | Description |
|---|---|---|
| "pagination" | Page | Paging and sorting of the result. |
| "from" | DateTime | Return entries from this UTC time. |
| "to" | DateTime | Return entries until this UTC time. |
| "severity" | String | Minimum log level to return. |
| "logger" | String | Logger (Java class) that wrote the entry; as filter, its name. |

## 3.6 struct Page

Paging and sorting; all fields optional.

| Object Field | Value Type | Description |
|---|---|---|
| "page" | Number | Zero-based page number. |
| "size" | Number | Number of records per page (at most `max.page.size`, default 1000). |
| "direction" | String | Sort direction, `ASC` or `DESC`. |
| "sortField" | String | Field to sort by. |

## 3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

| Type | Description |
|---|---|
| Array | A JSON Array [4]; Array<A> has elements of type A. |
| DateTime | JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`. |
| ExceptionType | One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String). |
| Number | JSON Number, an integer. |
| Object | JSON Object; Object<A> has string keys and values of type A. |
| String | JSON String. |

# 4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, dynamicServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

# 5 Revision History

## 5.1 Amendments

| No. | Date | Version | Subject of Amendments | Author |
|---|---|---|---|---|
| 1 | 2026-10-22 | 5.2.2 | Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace. | Jerker Delsing |

## 5.2 Quality Assurance

| No. | Date | Version | Approved by |
|---|---|---|---|
| 1 | | 5.2.2 | |